Лабораторная работа №12

Тема: Управление правами доступа в PostgreSQL

Цель: Получить практические навыки создания групп пользователей и изменения прав доступа для них.

Объем: 4 академических часов (1 неделя).

Задание

1. Используя pgAdmin4, создать новую роль пользователей **MyUser**.
   Опции: Лимит подключений 1, разрешить логин.
   Зайти под этим логином через консоль psql.
2. Выдать права на все таблицы в схеме public. Выполнить для них SELECT в консоли psql. Убедиться в работоспособности роли.
3. Отозвать права на одну из таблиц. Выполнить для неё SELECT в консоли psql. Убедиться в невозможности выполнения запроса.
4. Создать роль без прав с названием **MyUser2** и в соответствии с индивидуальными заданием согласно табл. 12.1 выдать права на необходимые действия для этой роли. Проверить работоспособность запросов.

Таблица 12.1. Индивидуальные задания на выдачу прав

№ варианта	Права
1	INSERT для таблицы
2	CONNECT к базе данных
3	EXECUTE функцию
4	DELETE для таблицы
5	DELETE ON ALL TABLES IN SCHEMA public WITH GRANT OPTION

6	SELECT для определённого поля таблицы WITH GRANT OPTION
7	ALL PRIVILEGES для типа данных
8	INSERT для определённого поля таблицы
9	UPDATE для таблицы
10	CREATE база данных
11	SELECT для таблицы
12	USAGE тип данных
13	ALL PRIVILEGES для таблицы WITH GRANT OPTION
14	UPDATE для определённого поля таблицы
15	EXECUTE ON ALL FUNCTIONS IN SCHEMA
16	CREATE SCHEMA
17	TRIGGER для таблицы
18	TRUNCATE ON ALL TABLES IN SCHEMA
19	ALL PRIVILEGES для функции
20	ALL PRIVILEGES для схемы

В отчет включить:

- скан-копию выполненных команд в psql, используя новую роль;
- sql команды выдачи/отзыва прав;
- все полученные результаты откомментировать по тексту отчета.

Методические указания к выполнению

Создание ролей пользователей

Для создания новой роли можно воспользоваться pgAdmin4, в окне добавления можно настроить дополнительные параметры, такие как: лимит подключений, срок годности, особые привилегии, разрешение на создание баз данных. Важно не забыть установить пароль, чтобы иметь возможность логина под этой ролью (рис. 12.1).

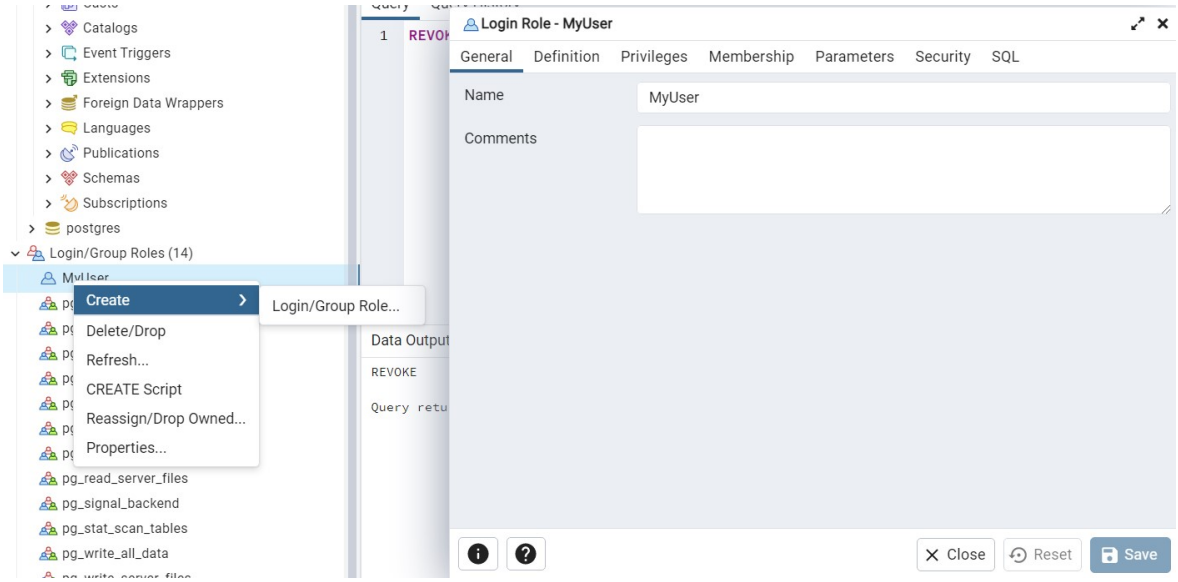

Рисунок 12.1 – Создание роли

После этого можно зайти, например, через консоль psql, и залогиниться под этой ролью. (рис. 12.2).

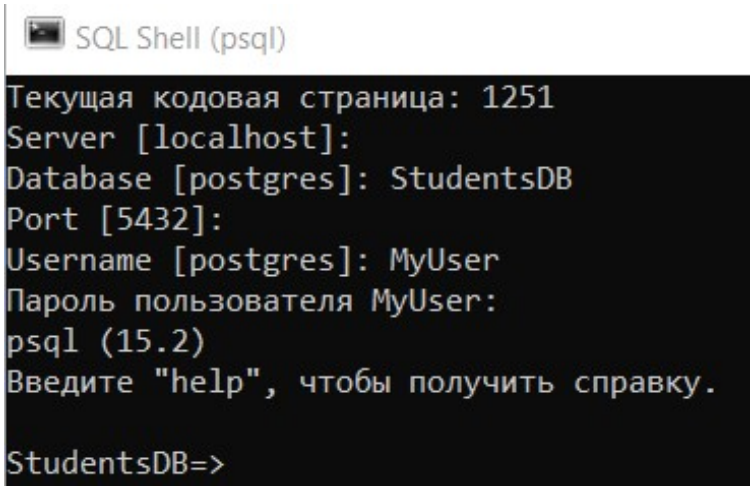

Рисунок 12.2 – Использование роли

По умолчанию у новой роли не будет прав доступа, если не установить флаг суперпользователя. Для того, чтобы выдать права, можно использовать SQL команду **GRANT**. При этом можно указать выдачу прав на конкретные действия, прописав ключевые слова SELECT | INSERT | UPDATE | DELETE | TRUNCATE | REFERENCES | TRIGGER. Либо же можно выдать сразу все права с помощью **ALL PRIVILEGES**. Права выдаются на конкретную

таблицу через **ON table**, либо же на всю схему сразу - **ALL TABLES IN SCHEMA name.** В конце команды указывается какой роли необходимо выдать права с помощью **TO role**. Дополнительно можно добавить **WITH GRANT OPTION**, чтобы разрешить передавать эти права другим ролям.

```
1   GRANT ALL PRIVILEGES ON public.sets TO "MyUser";
```

Если после этого попробовать выбрать данные из таблиц, то получим результат только для этой таблицы:

```
StudentsDB=> select * from sets;
Текущая кодовая страница: 1251
 setid | setname | setyear | specialtyid
-------+---------+---------+------------
     1 | ПИ-15   |    2015 |           1
     2 | ПИ-20   |    2020 |           2
     3 | ФИИТ-15 |    2015 |           3
     5 | ФИИТ-18 |    2018 |           3
     6 | ФИИТ-19 |    2019 |           3
     4 | ФИИТ-16 |    2016 |           3
    23 | test    |    2019 |           1
(7 строк)


StudentsDB=> select * from groups;
ОШИБКА:  нет доступа к таблице groups
StudentsDB=>
```

Теперь можно попробовать выдать права для всех таблиц в public схеме:

```
1   GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO "MyUser";
```

```
StudentsDB=> select * from groups;
ОШИБКА:  нет доступа к таблице groups
StudentsDB=> select * from groups;
Текущая кодовая страница: 1251
 groupid | setid | groupname | teachformid | isdeleted
---------+-------+-----------+-------------+-----------
       3 |     2 | ПИ-20-а   |           1 | f
       4 |     2 | ПИ-20-б   |           1 | f
       7 |     4 | ФИИТ-16-а |           1 | f
       8 |     4 | ФИИТ-16-б |           1 | f
       9 |     5 | ФИИТ-18-а |           1 | f
      10 |     5 | ФИИТ-18-б |           1 | f
      11 |     5 | ФИИТ-18-в |           2 | f
      12 |     6 | ФИИТ-20-а |           1 | f
      13 |     6 | ФИИТ-20-б |           1 | f
      14 |     6 | ФИИТ-20-в |           2 | f
      15 |     1 | ПМ-20-1   |           2 | f
      16 |     1 | ПМ-20-2   |           2 | f
(12 строк)
```

В результате появилась возможность работать другими таблицами.

Для того чтобы забрать определенные права у роли используется команда **REVOKE**. Она имеет аналогичные параметры как у команды **GRANT**. Можно указать конкретные действия либо убрать все привилегии как для одной таблицы, так и для всех таблиц на схеме. Главное отличие – в конце указывается вместо **TO** (кому выдаются права) ключевое слово **FROM** (у кого они забираются):

```
1  REVOKE ALL PRIVILEGES ON public.sets FROM "MyUser";
```

```
StudentsDB=> select * from sets;
ОШИБКА:  нет доступа к таблице sets
```

Дополнительно можно сразу после **REVOKE** и до определения всех параметров указать **GRANT OPTION FOR**. Такая команда не забирает указанные права у этой роли, однако таким образом она запрещает пользователю, находящемуся под этой ролью, выдавать такие права другим ролям (если у его роли изначально была добавлена возможность делать это).

Выдача и отзыв прав может быть выполнена не только для таблиц, но также и для процедур/функций, отдельных баз данных или даже на использование конкретных типов данных:

```
GRANT EXECUTE ON FUNCTION имя_функции
GRANT CREATE | CONNECT ON DATABASE имя_бд
GRANT USAGE ON TYPE имя_типа
```